Micropin ROM4 Shared-RAM Mailbox

A practical bridge between Jeri's five-socket RP2040 EPROM emulator and a modern game controller.

The key idea

Use the emulator's fifth chip-select input, ROM4, as a small memory-mapped shared-RAM window instead of a fifth read-only 2716 image. The original 8085 stays in place and runs a small hardware-agent ROM. A Pico W, Raspberry Pi, or other modern controller owns the game rules and exchanges commands/events through the shared window.

This replaces neither the 8085 nor the motherboard's port logic. It gives the modern side a dependable mailbox while preserving the real CPU's normal RAM and I/O cycles.

System architecture

```
Modern game process
| Wi-Fi / USB / serial
Controller + RP2040 mailbox service
| ROM4-selected shared-RAM window
8085 hardware-agent ROM
| ordinary STA / LDA / IN / OUT instructions
Micropin motherboard (RAM, ports, drivers, displays)
```

Why ROM4 is useful

Existing emulator capability	Mailbox extension
ROM0-3: five selectable 2 KB images stored in RP2040 flash.	Keep ROM0-3 as boot/agent ROM. Treat ROM4 as volatile RAM rather than flash-backed ROM.
/CE selects a 2 KB EPROM-like window.	Map the spare decoder output to an unused CPU memory region; even a 1 KB reachable portion is enough.
Data bus is driven on a selected read.	For a write, RP2040 releases the data bus and captures the byte supplied by the 8085.

Address-map hypothesis to verify

The PPM program occupies 0x0000-0x1FFF and demonstrably uses RAM through 0x23FE. If the existing RAM decode is only 1 KB, a spare select may expose a window such as 0x2400-0x27FF. This is a hypothesis, not a board fact: trace or scope the decoder before connecting the Pico.

The ROM4 device can physically contain 2 KB, but a decoder might expose only 1 KB of it. That is acceptable for a mailbox.

Required emulator changes

Signal / feature	Purpose
8085 /WR to a spare GPIO	Distinguish an 8085 write from a read while ROM4 is selected. GP25, now used only for heartbeat, is a plausible candidate in Jeri's current pin map.
Bidirectional data translation	During a CPU write, RP2040 outputs must be high impedance and the data byte must be visible to RP2040 inputs. During a read, the translator must let RP2040 drive the motherboard bus.
RAM buffer in RP2040 SRAM	A byte array for ROM4 contents. It should not be flash-programmed as part of normal mailbox traffic.
Fast read/write cycle handling	The present tight polling loop serves reads. Writes must be captured reliably at the actual 8085 bus rate; PIO is a strong candidate if C polling is marginal.
Safe USB-update behavior	A ROM-image USB update currently resets the target. Define what happens to volatile mailbox RAM and make the agent recover cleanly after reset.

ROM4 bus behavior

```
ROM4 selected, /WR high: RP2040 looks up mailbox[address] and drives D0-D7
ROM4 selected, /WR low: RP2040 releases D0-D7 and stores CPU-supplied byte
ROM4 not selected: RP2040 never drives the shared data bus
```

A separate /RD signal is desirable for diagnostic clarity, but may not be necessary if ROM4 /CE is only active for memory cycles and /WR cleanly distinguishes writes. Confirm this with a logic analyzer before simplifying the hardware.

Mailbox ownership rules

A mailbox is reliable when no byte has two writers. Each queue has one producer and one consumer; payload bytes are written first and a sequence/index byte is written last to commit the message.

Area	Writer	Reader	Use
Host-to-8085 command queue	Modern controller	8085 agent	Commands such as set score, set lamp state, pulse coil, play sound.
8085-to-host event queue	8085 agent	Modern controller	Switch changes, acknowledgments, error/status reports.
State snapshot	Defined field-by-field	Both	Diagnostics, heartbeat, requested vs. applied outputs.
Reserved / version bytes	Modern controller at boot	8085 agent	Protocol version and reset/reconnect detection.

Proposed 1 KB layout

```
0x2400-0x240F header: signature, protocol version, flags, heartbeat
0x2410-0x247F host-to-8085 ring: command packets and producer/consumer indices
0x2480-0x24EF 8085-to-host ring: events, responses, and indices
0x24F0-0x253F state snapshot: score/lamp/coil/sound diagnostics
0x2540-0x27FF reserved for expansion, trace capture, or larger queues
```

The literal base address will depend on the real decoder. The layout is intentionally relative: a 1 KB window can be placed anywhere the board offers a non-overlapping ROM4 select.

8085 agent responsibilities

Incoming modern command	8085 agent action
SET_SCORE or WRITE_DISPLAY_STATE	Validate fields; write the original physical RAM values required by any display/PROM logic.
SET_LAMP / PLAY_TONE	Translate to the known motherboard output ports with ordinary OUT instructions.
PULSE_COIL	Enforce a coil whitelist and maximum duration; force an off state on timeout or watchdog failure.
READ_SWITCHES	Read existing input ports and publish a compact event/state response into the mailbox.
HEARTBEAT missing	Enter a safe state: reject new coil actions and turn off controlled outputs as the hardware design permits.

What the mailbox does not do

Writing a lamp or coil value into ROM4 RAM does not itself control the motherboard. The 8085 agent must consume that command and issue the native output-port write. Likewise, direct Pico writes to the original motherboard RAM are not part of this first design; they would require HOLD/HLDA bus arbitration and are unnecessary for the mailbox approach.

Bring-up sequence

Step	Evidence of success
1. Decode survey	Scope/logic-analyzer capture proves ROM4's memory range and absence of overlap with active RAM.
2. Read-only ROM4 test	8085 LDA from ROM4 returns a predictable pattern with no bus contention.
3. Write/readback test	8085 STA then LDA proves RAM-mode capture and data-bus direction control.
4. Polling protocol	8085 and host exchange sequence-numbered bytes, then ring-buffer packets.
5. Passive outputs	Add display/lamp actions first; inspect behavior under reset and controller disconnect.
6. Controlled coils	Enable only with duration limits, watchdog handling, and measured safe defaults.

Design conclusion

ROM4-as-mailbox is a low-intrusion bridge: it uses an existing emulator concept and a spare memory decode rather than requiring a new I/O decoder or an 8085 replacement. Its first risk-reduction milestone is simple and concrete: prove an 8085 write/read round trip through the ROM4 window on the real Micropin bus.